BINUS UNIVERSITY ONLINE LEARNING

People Innovation Excellence

LECTURE NOTE

Week 08

String Matching

ALGORITHM DESIGN AND ANALYSIS

LEARNING OUTCOMES

LO2: compare several algorithm design methods

OUTLINE MATERI :

1. Introduction to String Processing
2. Naïve String-Matching Algorithm
3. The Knuth-Morris-Pratt Algorithm

ISI MATERI

1. Introduction to String Processing

Komputer sebelumnya hanya digunakan untuk memproses data numerik. Selanjutnya, ketika komputer berevolusi, mereka digunakan untuk memproses data non-numerik seperti karakter dan string. Karakter atau simbol dapat berupa huruf, literal, atau tanda baca, dan bergantung pada domain aplikasi. String adalah sekumpulan karakter. String dibangun dari satu set nol atau lebih karakter. Beberapa aplikasi yang melibatkan pemrosesan string, yaitu:

Text editing: utilitas perangkat lunak populer seperti Microsoft Word dan OpenOffice untuk membuat dokumen adalah contoh yang baik dari pemrosesan string. Seseorang dapat mengamati bahwa algoritma string seperti menyalin string, menghapus, mengedit, menyelaraskan teks, dan algoritma matching pattern sering digunakan dalam perangkat lunak desktop publishing.

Spell checking: Sering kali, paket pengolah kata yang populer mengidentifikasi kemungkinan kesalahan ketik dalam kata-kata dan menyarankan kemungkinan alternatif kata yang benar. Ini adalah operasi klasik yang menggunakan algoritma pengeditan string untuk menemukan kata-kata yang lebih tepat.

Plagiarism checking: Plagiarisme adalah penyalinan konten orang lain tanpa izin. Pemeriksaan plagiarisme mungkin melibatkan pemeriksaan dua dokumen ganda untuk kesamaan konten.

Similarity checking: Pemeriksaan kesamaan adalah proses memeriksa tingkat kesamaan antara dua dokumen. Idenya dapat diperluas untuk membandingkan dua program dan digunakan dalam rekayasa perangkat lunak untuk kontrol versi.

Computational biology: Dalam biologi komputasi, urutan DNA dianggap sebagai data string. DNA merupakan polimer dan direpresentasikan sebagai polinukleotida yang terdiri dari adenin (A), guanin (G), sitosin (C), dan timin (T). Urutan DNA yang khas adalah sebagai berikut: ACTAGGGGT. Aplikasi khas biologi komputasi yang menggunakan algoritma string termasuk menyimpulkan hubungan keluarga, perumusan evolusi pohon, identifikasi generasi tertentu, dll.

Terminologi berikut cukup berguna untuk memahami manipulasi string:

Alphabet. Alfabet adalah kumpulan karakter dasar yang digunakan untuk berkomunikasi.

String. Urutan karakter berhingga yang diambil dari alfabet disebut string.

Length. Jumlah karakter dalam string

Null string. Sebuah string ∈ dengan panjang 0

Suffix. Biarkan s menjadi string. Dengan menghapus nol atau lebih karakter di awal string s, seseorang mendapatkan string yang disebut sufiks

Prefix. Awalan dibentuk dengan menghapus nol atau lebih karakter dari akhir string s.

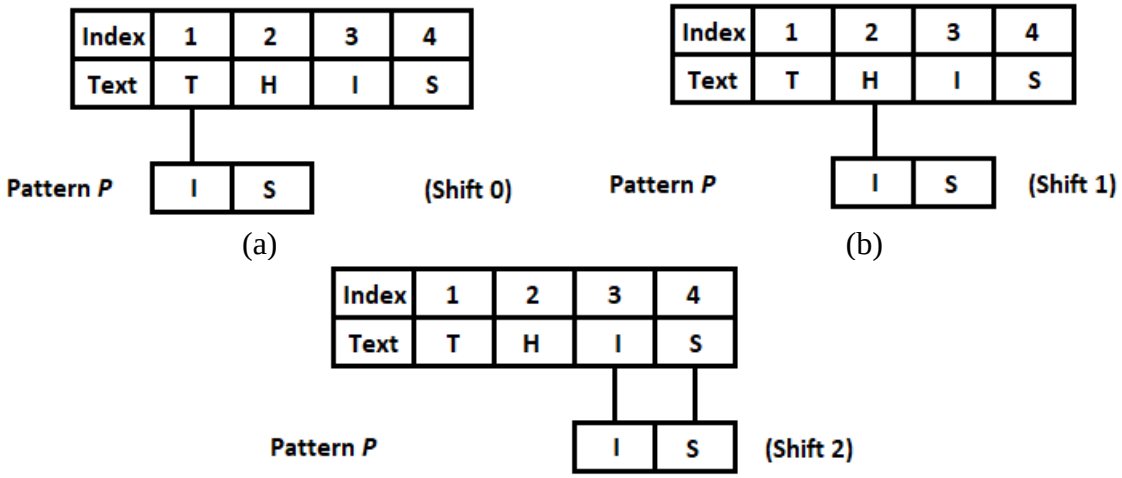
String matching adalah suatu metode yang digunakan untuk mencocokan symbol atau huruf dalam suatu deret. Pada metode ini akan dipelajari Bagaimana computer bisa mendapatkan satu pola atau deretan pola yang diberikan. Ide dasarnya muncul dari permasalahan umum yaitu Bagaimana mendapat symbol atau huruf dalam kumpulan teks yang diberikan. Pembahasan mencakup berbagai teori Dasar dari algoritma pencarian dan pencocokkan simbol atau string antara lain: naïve string matching dan algoritma Knuth-Morris-Pratt

2. Naïve String Matching

Algoritma naïve string matching (atau pencocokan pola) dapat dinyatakan sebagai berikut: Dalam teks T dan pola P yang diberikan, masalah pencocokan pola adalah untuk memeriksa apakah P ada di T atau tidak, dan juga di mana P muncul di T.

Algoritma naïve string matching adalah contoh dari algoritma brute force. Algoritma ini menerima teks (panjang *n*) dan pola (panjang *m*), dan memeriksa *m* karakter dari posisi pertama teks. Jika terjadi ketidaksesuaian, pola digeser satu posisi. Di sisi lain, jika ada kecocokan, maka karakter berikutnya dicentang. Dengan kata lain, pergeseran, katakanlah *k*, adalah offset yang diperlukan untuk menyelaraskan karakter pertama dari pola dengan karakter *k+1* teks *T*. Pergeseran *k* yang memberikan pencocokan tepat disebut pergeseran yang valid. Proses

pencarian ini berlanjut sampai pola ditemukan atau teks benar-benar habis. Perhatikan contoh naïve string matching dengan teks T = 'THIS' dan pola P ='IS', dan hasil antara ditunjukkan pada Gambar 16.5.



Gambar 1. Naïve pattern matching (a) Pattern *P* and text; shift 0 (b) Move the pattern: shift 1; no match between index 2 of *T* and *P* (c) Move to pattern; shift is 2; match index 3 and then 4 of *T* and *P*; report success.

Algoritma naïve string matching informal diberikan sebagai berikut:

langkah 1: Baca teks T dan pola P

langkah 2: Sejajarkan pola P dengan teks T, dan pindah dari kiri ke kanan

langkah 3: Ulangi langkah-langkah berikut sampai seluruh pola ditemukan atau akhir teks T ditemukan

> **3a:** Bandingkan karakter pertama teks T dan pola P. Jika ada kecocokan, maka pindah untuk membandingkan karakter kedua, dan seterusnya. Jika semua karakter P cocok dengan karakter T, laporkan sukses dan keluar; jika tidak, lanjutkan ke Langkah 4
>
> **3b:** Pindahkan pola ke bawah teks dengan satu karakter (atau dengan shift) dan tambahkan nomor shift; pergi ke Langkah 3

langkah 4: Jika pola P tidak ada di Langkah 3 dan 4, laporkan kegagalan dan akhiri.

Kelebihan dan Kekurangan Algoritma naïve string matching.

Keuntungan dari algoritma naïve string matching adalah sebagai berikut:

1) Algoritma tidak memerlukan pra-pemrosesan apa pun
2) Tidak memerlukan ruang tambahan.
3) Dapat digunakan sebagai program benchmark untuk membandingkan kinerja

Kerugian utama dari algoritma pencocokan pola naif adalah sebagai berikut:

1) Karena masalah praktis melibatkan teks dan pola yang lebih besar, algoritma naïve string matching tidak cocok
2) Jika ada banyak contoh pola dalam sebuah teks, teknik ini tidak berguna
3) Algoritma naïve pattern matching sangat lambat.

3. Knuth – Morris – Pratt Algorithm

Algoritma pola pencocokan tepat linier untuk pencocokan string cepat ini dirancang oleh Donal E. Knuth, James H. Morris, dan Vaughan R. Pratt pada tahun 1977. Tujuan dari algoritma ini adalah untuk meminimalkan jumlah total perbandingan antara pola *P* dan teks *T* Misalkan pola *P* dengan panjang *m* dan teks *T* dengan panjang *n*, dengan $n > m$. Algoritma Knuth-Morris-Pratt (singkatnya KMP) lebih efisien daripada algoritma *naïve string matching*. Pertimbangkan situasi yang ditunjukkan pada Gambar 2.

Index:	1	2	3	4	5	6	7
Text:	A	B	A	A	B	A	B
Pattern:	A	B	A	B			

Gambar 2. Text T and pattern P

Algoritma *naïve string matching* berjalan dengan cara berikut: karena ketidakcocokan terjadi pada $P[4] \neq T[4]$, algoritme menggeser pola satu karakter. Namun, ini tidak efektif, karena $P[1]$ tidak sama dengan $T[2]$. Alih-alih, pergeseran yang lebih baik adalah yang ditunjukkan pada Gambar 3.

Index:	1	2	3	4	5	6	7
Text:	A	B	A	A	B	A	B
Pattern:			A	B	A	B	

Gambar 3. Better shift of pattern

Dengan kata lain, seseorang harus mencari lebih dari satu shift, karena jelas bahwa shift yang lebih baik dapat menghindari perbandingan yang tidak perlu. Informasi tentang pergeseran yang lebih baik ini dapat dengan mudah diperoleh dari pola itu sendiri. Algoritma KMP dengan demikian mengekstrak informasi tentang pergeseran yang lebih baik berdasarkan pola itu sendiri dan merupakan fokus diskusi di bagian ini. Algoritma KMP bekerja dalam dua tahap. Pada tahap 1, pola tersebut diproses terlebih dahulu untuk mendapatkan informasi sehingga perbandingan sepele dapat dikurangi dan, pada tahap 2, menggunakan informasi ini untuk menentukan pergeseran yang lebih baik untuk pencocokan pola yang efektif. Mari kita bahas tahap 1 sekarang.

Tahap 1

Tujuan dari tahap pertama adalah untuk melakukan praproses pola untuk menemukan fungsi awalan yang dapat membantu melewatkan karakter yang tidak perlu selama proses pencocokan. Mari kita asumsikan bahwa ketidakcocokan terjadi pada posisi *j* teks, yaitu T_j dan pada posisi P_{k+1} dari pola yang diberikan; sekarang, awalan $P_1, P_2, \ldots, P_k$ sama dengan substring P_{j-k}, P_{j-k-1},..., P_{j-1} dari teks *T*. Untuk memahami ini, mari kita ingat kembali beberapa hal penting definisi tentang awalan dan akhiran yang tepat. Awalan yang tepat dibentuk dengan menghapus nol atau lebih karakter dari akhir. Misalnya, m, ma adalah semua awalan dari string 'man'. Demikian pula, sufiks yang tepat dibentuk dengan menghapus nol atau lebih karakter dari awal. Misalnya, mage, age, ge, dan e adalah sufiks yang tepat dari string 'image' . String kosong adalah awalan dan akhiran dari sebuah string.

Singkatnya, algoritma KMP menemukan awalan yang tepat terpanjang dalam subpola yang cocok dengan akhiran yang tepat dalam string yang sama, dan informasi berharga ini disimpan dalam tabel yang disebut tabel shift. Secara umum, ini dinyatakan secara matematis sebagai berikut:

$$f(c) = max\{k, k < c \text{ and } P_1, P_2, \ldots, P_k \text{ is prefix of } P_c\}$$

Dengan kata lain, $f(c)$ menunjukkan awalan terpanjang dari *P* yang cocok dengan akhiran P_c. juga dapat diamati bahwa f adalah fungsi awalan yang memetakan *f*(1...*m*-1) ke fungsi *f*(0...*m*-1). Untuk mengilustrasikannya, mari kita perhatikan contoh sederhana dari string 'XYX'. Apa yang harus menjadi entri di tabel shift? Untuk menemukan awalan yang tepat terpanjang yang cocok dengan awalan yang tepat dari XYX adalah X dan Y. Demikian pula, akhiran yang tepat adalah Y dan X. Oleh karena itu, awalan terpanjang yang cocok dengan akhiran yang tepat adalah 'X'. Panjangnya adalah 1. Oleh karena itu, entri string ini adalah 1.

Contoh:

Temukan perhitungan awalan string 'XYXRXY'

Penyelesaiannya:

Masalah yang diberikan dapat diselesaikan dengan menggunakan dua tahap algoritma KMP. Tahap 1 dari algoritma KMP adalah pra-pemrosesan pola untuk menemukan fungsi awalan. Mulailah membaca karakter dari kiri ke kanan satu per satu. X ditemui pertama kali. Oleh karena itu, $f(0) = 0$. Kemudian baca Y. Ini juga ditemui untuk pertama kalinya. Oleh karena itu, $f(1) = 0$. baca karakter X. X telah ditemukan. Oleh karena itu, $f(2) = 1$. Dapat dicatat bahwa X adalah akhiran dari 'XYX'. Baca karakter berikutnya R. Ini ditemui pertama kali. Oleh karena itu, $f(3) = 0$. Sekali lagi $f(4)$ adalah 1, karena X adalah akhiran dari string 'XYXRX', dan akhirnya $f(5)$ adalah 2, karena string XY adalah akhiran dari string 'XYXRXY '. Fungsi awalan tercantum dalam tabel 1. Tabel ini digunakan untuk menunjukkan pergeseran yang diperlukan yang diperlukan untuk pencocokan lebih cepat.

Table 1. Prefix function

Index(*j*)	0	1	2	3	4	5
Characters	X	Y	X	R	X	Y
F(*j*)	0	0	1	0	1	2

Tahap 2

Tahap 2 dari algoritma KMP adalah bagian yang cocok. Setelah perhitungan awalan dilakukan, itu disimpan dalam tabel shift. Algoritma KMP berkonsultasi dengan tabel untuk menemukan pergeseran yang efektif. Mari kita asumsikan bahwa *c* adalah panjang karakter yang cocok. Jika $c = m$, maka masalah terpecahkan. Jika tidak, pergeseran akan menjadi sebagai berikut: panjang pencocokan pola - entri tabel pergeseran di lokasi $c - 1$. Berdasarkan ini, pola digeser melintasi teks untuk pencocokan.

Dengan demikian, algoritma KMP yang menggabungkan tahap pencocokan dapat dinyatakan secara informal sebagai berikut:

Langkah 1: Temukan panjang pola dan teks:

$$n = |T|$$

$$m = |P|$$

Langkah 2: Terapkan preprocessing dan temukan fungsi awalan *f*

Langkah 3: Biarkan *c* menjadi jumlah karakter yang cocok sejauh ini.

$$c = 0$$

Langkah 4: bandingkan karakter pertama pola *P* dengan karakter pertama teks *T*. Jika ada kecocokan, maka increment *c*; jika tidak, *c* diperbarui menggunakan tabel perhitungan awalan.

Langkah 5: Ulangi langkah 4 hingga semua karakter pola *P* cocok dengan teks *T*. Jika cocok, laporkan keberadaan pola dan cetak pergeserannya; lain ulangi proses pencarian.

Langkah 6: cari kecocokan berikutnya jika perlu; jika tidak, akhiri.

Contoh:

Bentuk perhitungan awalan untuk pola P = 'aab' dan cocokkan dengan string 'aadbaaba' menggunakan prosedur KMP

Penyelesaiannya:

Untuk menerapkan algoritma KMP, terlebih dahulu perlu dilakukan perhitungan prefix. Fungsi kegagalan untuk string 'aab' dihitung dan ditunjukkan pada tabel 16.3

Index(*j*)	0	1	2
Characters	a	a	b
F(*j*)	0	1	0

Sekarang, f(0) adalah 0, karena string hanya a; *f*(1) adalah 1, karena a adalah akhiran dari string 'aa'. Sekali lagi, *f*(2) adalah 0, karena b bukan akhiran dari string 'aab'. Pencocokan string ditunjukkan pada Gambar. 16.12. (a) - (e)

Knuth-Morris-Pratt Algorithm							
a	a	d	b	a	a	b	a
a	a	b					
Comparisons = 0							

(a)

Knuth-Morris-Pratt Algorithm							
a	a	d	b	a	a	b	a
	a	a	b				
Comparisons = 3							

(b)

Knuth-Morris-Pratt Algorithm							
a	a	d	b	a	a	b	a
			a	a	b		
Comparisons = 4							

(c)

Knuth-Morris-Pratt Algorithm							
a	a	d	b	a	a	b	a
				a	a	b	
Comparisons = 5							

(d)

Knuth-Morris-Pratt Algorithm									
a	a	d	b	a	a	b	a		
					a	a	b		
Comparisons = 8									

(e)

Gambar 4 KMP matcher (a) initial position; there is a mismatch between ‘d’ dan ‘b’ (b) Move the pattern; there is a mismatch between, ‘d’ and ‘a’ (c) move the pattern there is a mismatch between ‘d’ and ‘a’ (d) Move the pattern; there is a match between all characters; report success (e) Move the pattern; text is eshauted. Stok search

KESIMPULAN

Komputer sebelumnya hanya digunakan untuk memproses data numerik. Selanjutnya, ketika komputer berevolusi, mereka digunakan untuk memproses data non-numerik seperti karakter dan string. Karakter atau simbol dapat berupa huruf, literal, atau tanda baca, dan bergantung pada domain aplikasi. String adalah sekumpulan karakter. String dibangun dari satu set nol atau lebih karakter. Beberapa aplikasi yang melibatkan pemrosesan string, yaitu: text editing, spell checking, plagiarism checking, similarity checking, computational biology.

String matching adalah suatu metode yang digunakan untuk mencocokan symbol atau huruf dalam suatu deret. Pada metode ini akan dipelajari Bagaimana computer bisa mendapatkan satu pola atau deretan pola yang diberikan. Ide dasarnya muncul dari permasalahan umum yaitu Bagaimana mendapat symbol atau huruf dalam kumpulan teks yang diberikan. Pembahasan mencakup berbagai teori Dasar dari algoritma pencarian dan pencocokkan lambing ataru string antara lain: naïve string matching dan algoritma Knuth-Morris-Pratt.

Algoritma naïve string matching (atau pencocokan pola) dapat dinyatakan sebagai berikut: Dalam teks T dan pola P yang diberikan, masalah pencocokan pola adalah untuk memeriksa apakah P ada di T atau tidak, dan juga di mana P muncul di T.

DAFTAR PUSTAKA

Sridhar, S. (2015). *Design and Analysis of Algorithms*. Oxford University Press. Chapter 16

https://www.javatpoint.com/daa-knuth-morris-pratt-algorithm

The Knuth-Morris-Pratt (KMP)Algorithm, https://www.javatpoint.com/daa-knuth-morris-pratt-algorithm

Search: Optimal, Branch and Bound, A*, www.youtube.com/watch?v=gGQ-vAmdAOI

Strings, www.youtube.com/watch?v=NinWEPPrkDQ